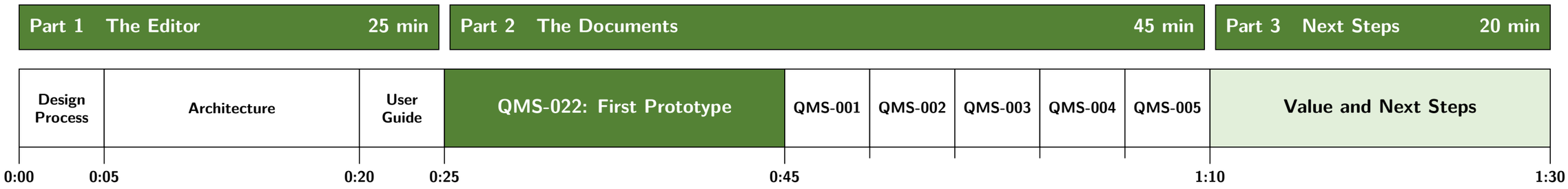
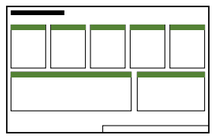
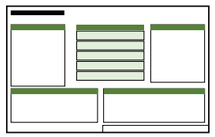
Agenda

QMS Editor review: the editor, then the six documents built with it. 1 hr 30 min.

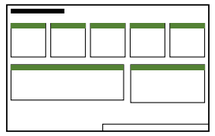




Sheet 2
Design Process
5 min



Sheet 3
Architecture
15 min



Sheet 4
User Guide
5 min

Centerpiece *the first prototype*

QMS-022

ECN Factory Implementation Process

Before and after *8 min*

The Word original next to the new PDF, with the process maps in Appendix A and B

Live edit *12 min*

1. Open `QMS-022_source.zip`
2. Change a field
3. **Build preview**
4. **Finish & create files**

Questions as we go

QMS-001 to QMS-005

5 min each



For each document

- Page through the PDF next to its Word original
- Note content corrections for the document owner
- Same template, checks and Revision History as QMS-022



Sheet 5
Value to Doosan Bobcat
5 min

Open discussion *15 min*

- Questions on the editor and the six documents
- Agree the next steps on sheet 5

We leave with: approval of the QMS-022 format and a plan to review and approve the remaining documents.

Title	Project	Version	Rev.	Date	Drawn	Sheet
Agenda	**QMS Editor**	**2.0**	**12**	**2026-10-23**	**Gust F.**	**1 of 5**

QMS Editor: Design Process

Planned against written requirements, built in stages, and tested on every document before it was handed over.

1 Defined the requirements

Before any code

- Took four project requirements from Gary R.: no cost, PDF finals, time-effective updates, repeatable at scale
- Worked through the brief's 13 detailed requirements, matching each to a feature
- Designed for non-technical users: no coding required, no Word formatting
- Kept documents and review in SharePoint

2 Studied the documents

All North America QMS documents

- Reviewed the Word originals one by one
- 28 of 30 follow the QMS-022 pattern
- So one template, one page design and one set of rules can serve all of them

3 Chose the platform

Criteria for a company PC

- Ruled out installers and administrator rights
- Packaged the app as one file to hand to anyone
- Built the PDF engine into the app, so every PC prints the same
- Used free, open-licence parts only

4 Built and verified

Prototype first, then the product

- Prototyped an HTML editor with a Python builder and checker
- Rebuilt it as a Windows desktop app
- Held the new engine to the prototype's output, character for character

5 Refined with feedback

Changes after review of version 1

- Removed the document library; the app became a focused `.tex` editor
- Redrew the turtle diagram: symmetric, straight arrows
- Made backend optimizations
- Made long lists scrollable

How it was proven

Test	What it checks	Result
Output match	New `.tex` writer against the original Python builder, every document	32 of 32 identical
Check match	Errors and warnings against the original checker	32 of 32, plus 16 of 16 broken on purpose
Round trip	Write a document to `.tex`, read it back, compare	32 of 32 exact
Older files	Files from the first editor read back and print the same	30 of 30 print identically
End to end	Setup, load, edit, preview, finish, save, upload to SharePoint	All checks pass
Screen sizes	Long Input and Output lists reachable at three screen sizes	Pass; fails on the old version
Pictures	PNG, JPG, PDF and SVG pictures reach the PDF	Pass
Code reviews	Independent review of the desktop code and SharePoint sign-in	Issues found and fixed
On a work PC	Real build, PDF engine, job-title lookup, turtle look	Confirmed

Proof of concept: QMS-022

ECN Factory Implementation Process

✓

Passed every check: zero errors, zero warnings, and it reopens exactly

- Converted to the new format and checked against the Word original
- Both process maps captured from the original at 300 dpi, not redrawn; connector lines keep their routes
- Spelling error in the original corrected ("Notificaiton")
- First Revision History row in the new format, dated 22 Oct 2026

System Architecture

Documents live in SharePoint. The app on the PC reads a `.tex` and writes a finished PDF and a source zip.

Where the files live

A SharePoint document folder

The controlled library

`QMS-ONN.pdf` the published document
`QMS-ONN_source.zip` kept for the next edit
Document control: reviewer, review cycle, reminders

B Each user's PC

The app and its working files; no documents

`QMS Editor.exe`
`%LOCALAPPDATA%\QMS-Editor` settings, PDF engine
`%LOCALAPPDATA%\QMS-TeX` package cache, once per PC (100–150 MB)

C Gust's OneDrive `TeX` folder

Build and backup

App source code and one-click build
`source\QMS documents (.tex)` converted documents
Word originals and the first editor, kept as backup

Input

Final .PDF & updated source files (.zip)

Desktop shell — *Rust, Tauri 2*

Files, zips, job-title lookup, SharePoint sign-in

Screens — *HTML, CSS, JavaScript in Edge WebView2*
Setup wizard, Home, editor, Finish dialog

Core engine — *JavaScript (`core.js`)*
Writes and reads `.tex`, runs the checks, sizes tables

`.tex` ↓ ↑ PDF

PDF engine — *Tectonic 0.15 (XeTeX), built in*
Compiles the `.tex` with the page design into the PDF

Page design — *`doosanqms.cls, turtle.tex, template`*
Layout, branding, header, footer, turtle diagram

PDF BUILD

Runs entirely on the PC; it reaches only SharePoint and the company directory.

Inside a source zip

`QMS-ONN_source.zip`

- `QMS-ONN.tex` — *The document: text and data*
- `maps\` — *Process-map pictures (PNG, JPG, PDF)*
- `doosanqms.cls` — *Page layout and branding*
- `turtle.tex` — *Turtle diagram*
- `brand\doosan-bobcat-logo.png` — *Header logo*

Everything needed to rebuild the PDF, in one file. Open it next time to keep editing.

Inside a `.tex` file

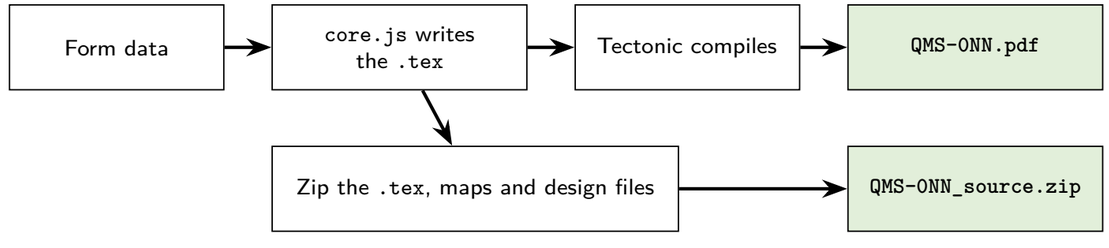
```
\documentclass{doosanqms}
...
\end{document}
```

The document. Any TeX engine builds the PDF from it.

```
%QMS-DATA-BEGIN
% the document's data
%QMS-DATA-END
```

The data, as comments. TeX ignores it; the app reads it back exactly, so nothing is lost between edits. Older files without it are read from the TeX.

How the PDF and zip are made



The preview takes the same path: what the user previews is what gets published.

Title	Project	Version	Rev.	Date	Drawn	Sheet
Architecture	**QMS Editor**	**2.0**	**12**	**2026-10-23**	**Gust F.**	**3 of 5**

User Guide

Five steps from a SharePoint file to a published PDF. The user edits content in a form; the app does the layout, the checks and the bookkeeping.

1 Set up

Wizard on first start, once per PC

1. Welcome
2. Name and job title, filled from the company directory
3. PDF engine
4. SharePoint folder
5. Done

2 Load

Three ways to load (table below)

- The file lands in the Input list
- Load several documents in one session
- Click **Edit** to open one

3 Edit

A form in the same order as the PDF

- Header, Purpose, Turtle diagram, Sections and tables, Appendices, Revision History
- Outline on the left, form in the middle, PDF preview on the right

4 Finish

Finish & create files

- Adds the Revision History row: Rev. Date, Revised by, Job Title, Summary of Changes
- Sets the header Rev Date
- Creates the PDF and source zip

5 Hand off

From the Output list

- Save both files to the PC, or upload both to SharePoint
- Review then runs in SharePoint document control, outside the app

Loading and saving

Load	
Upload Current Document (.tex)	Pick or drag in a `.tex`, or a `_source.zip` to bring its pictures too
Pick from the SharePoint folder	Lists the folder's `.tex` and `_source.zip` files; the user signs in with their own Microsoft account
Create a new document	Enter the document ID, title and owner; starts from the standard template
Choose picture...	Adds a process map or figure from Visio, PowerPoint and similar: PNG, JPG, PDF or SVG
Save	
Save to this PC	The PDF and the source zip to a folder the user picks
Upload to SharePoint	Both files to the document folder; asks before replacing an existing file

What the editor takes care of

The user supplies content; the rest is automatic

- **Layout.** Fonts, branding, spacing and page breaks are fixed by the page design; the user cannot drift from the standard.
- **Error checking.** Errors and warnings in plain English, shown at the field while the user types.
- **Fast feedback.** **Build preview** shows the PDF in a few seconds and jumps to the page being edited.
- **Simple entry.** The turtle diagram is one line per bullet; tables are rows of cards, with column widths sized to the text.

Value to Doosan Bobcat

Every finished document is a standard, branded PDF, produced with tools the company already owns or that are free.

Cost

$0

licence cost to roll out, and per document created

- Free, open-licence parts: Tectonic (MIT), TeX Live packages (LPPL), Tauri and Rust (MIT/Apache), Carlito font (OFL)
- Runs on what the company already pays for: Windows, Microsoft 365, SharePoint, Calibri
- No subscriptions, cloud services, AI services or paid add-ins
- The only cost is staff time to write the content

Consistency and control

PDF only

every final version, in one standard look

- Controlled-document elements on every page: header with Title, Owner, Location, Section, Rev Date and Page x of y; the three footer statements
- Revision History on every document
- New Doosan logo in Endeavour Blue, with the current Bobcat logo
- Process maps print exactly as supplied, at full resolution

Scale and continuity

30

North America QMS documents already converted

- The library grows in SharePoint, not in the app
- One `.exe` to share with another person or function
- No lock-in: any TeX engine rebuilds a document from its source zip, even without the app
- Versioned data format; older files upgrade on opening

Proposed next steps

1. Review the QMS-022 proof of concept with Gary R. and approve the format.
2. Capture the process maps for the remaining documents from the Word originals.
3. Review and approve the translated documents by hand.
4. Publish each document through SharePoint document control.
5. Continue to maintain `QMS Editor.exe` and the documents after each ISO audit.

Title	Project	Version	Rev.	Date	Drawn	Sheet
Value	**QMS Editor**	**2.0**	**12**	**2026-10-23**	**Gust F.**	**5 of 5**